二分算法是我觉得在基础算法篇章中最难的算法。二分算法的原理以及模板其实是很简单的，主要的难点**在于问题中的各种各样的细节问题**。因此，大多数情况下，只是背会二分模板并不能解决题目，还要去处理各种乱七八糟的边界问题。

【案例】

> **题目来源：** 力扣
>
> **题目链接：** **34. 在排序数组中查找元素的第一个和最后一个位置**
>
> **难度系数：** ★★

【题目描述】

> 给你一个按照非递减顺序排列的整数数组 `nums`，和一个目标值 `target`。请你找出给定目标值在数组中的开始位置和结束位置。
>
> 如果数组中不存在目标值 `target`，返回 `[-1, -1]`。
>
> 你必须设计并实现时间复杂度为 `O(log n)` 的算法解决此问题。

【算法原理】

当我们的解具有**二段性**时，就可以使用二分算法找出答案：

- 根据待查找区间的中点位置，分析答案会出现在哪一侧；
- 接下来舍弃一半的待查找区间，转而在有答案的区间内继续使用二分算法查找结果。

【模板】

二分的模板在网上至少能搜出来三个以上。但是，我们仅需掌握一个，并且一直使用下去即可。

代码块

```
// 二分查找区间左端点
int l = 1, r = n;
while(l < r)
{
    int mid = (l + r) / 2;
    if(check(mid)) r = mid;
    else l = mid + 1;
}
// 二分结束之后可能需要判断是否存在结果
```

代码块

```
// 二分查找区间右端点
int l = 1, r = n;
while(l < r)
{
    int mid = (l + r + 1) / 2;
    if(check(mid)) l = mid;
    else r = mid - 1;
}
// 二分结束之后可能需要判断是否存在结果
```

> 为了防止溢出，求中点时可以下面的方式：

- `mid = l + (r - l) / 2`

【时间复杂度】

每次二分都会去掉一半的查找区域，因此时间复杂度为 $logN$ 。

【模板记忆方式】

1. 不用死记硬背，算法原理搞清楚之后，在**分析题目的时候**自然而然就知道要怎么写二分的代码；
2. 仅需记住一点， $if/else$ 中出现 -1 的时候，求 mid 就 $+1$ 就够了。

【二分问题解决流程】

1. 先画图分析，确定使用左端点模板还是右端点模板，还是两者配合一起使用；
2. 二分出结果之后，不要忘记判断结果是否存在，二分问题细节众多，一定要分析全面。

【STL 中的二分查找】

`<algorithm>`

1. `lower_bound` ：大于等于 x 的最小元素，返回的是迭代器；时间复杂度： $O(\log N)$ 。
2. `upper_bound` ：大于 x 的最小元素，返回的是迭代器。时间复杂度： $O(\log N)$ 。

二者均采用二分实现。但是 STL 中的二分查找只能适用于"在有序的数组中查找"，如果是二分答案就不能使用。因此还是需要记忆二分模板。

7.1 二分查找

7.1.1 牛可乐和魔法封印

题目来源： 牛客网

题目链接： 牛可乐和魔法封印

难度系数： ★★

【题目描述】

牛可乐得到了一个长度为 n 且非严格单调递增的序列 a ，然而这个序列被 q 层魔法封印了，其中第 i 层封印的问题包含两个整数 $x_i, y_i (x_i \leq y_i)$ ，牛可乐必须正确回答序列中大于等于 x_i 且小于等于 y_i 的数字个数才能够解开该层封印。

牛可乐觉得这个问题太难了，于是他想请你帮助他解开序列的 q 层封印。

【输入描述】

第一行包含一个整数 $n(1 \leq n \leq 10^5)$，表示序列的长度
第二行包含 n 个整数，其中 $-10^9 \leq a_i \leq 10^9$
第三行包含一个整数 $q(1 \leq q \leq 10^5)$，代表封印层数。
之后 q 行，每行两个整数 $x_i, y_i(-10^9 \leq x_i \leq y_i \leq 10^9)$，代表该层封印的询问。

【输出描述】

对于每层封印，输出一行一个整数，代表在范围内的数字个数。

【示例一】

输入：

```
5
1 2 3 4 5
3
2 6
1 5
3 3
```

输出：

```
4
5
1
```

【示例二】

输入：

```
5
1 1 2 3 3
3
1 1
2 3
1 3
```

输出：

```
2
3
```

```
5
```

【解法】

二分查找算法模版题，直接上手模版即可。

但是需要注意，有可能并没有这个区间，需要在二分结束之后判断一下。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];

int binary_search(int x, int y)
{
    // 大于等于 x 的最小元素
    int left = 1, right = n;
    while(left < right)
    {
        int mid = (left + right) / 2;
        if(a[mid] >= x) right = mid;
        else left = mid + 1;
    }
    if(a[left] < x) return 0;
    int tmp = left;

    // 小于等于 y 的最大元素
    left = 1, right = n;
    while(left < right)
    {
        int mid = (left + right + 1) / 2;
        if(a[mid] <= y) left = mid;
        else right = mid - 1;
    }
    if(a[left] > y) return 0;

    return left - tmp + 1;
}

int main()
```

```
37  {
38      cin >> n;
39      for(int i = 1; i <= n; i++) cin >> a[i];
40
41      int Q; cin >> Q;
42      while(Q--)
43      {
44          int x, y; cin >> x >> y;
45          cout << binary_search(x, y) << endl;
46      }
47
48      return 0;
49  }
```

7.1.2 A-B 数对

题目来源： 洛谷

题目链接： A-B 数对

难度系数： ★

【题目描述】

给出一串正整数数列以及一个正整数 C，要求计算出所有满足 $A-B=C$ 的数对的个数（不同位置的数字一样的数对算不同的数对）。

【输入描述】

输入共两行。

第一行，两个正整数 N, C。

第二行， N 个正整数，作为要求处理的那串数。

【输出描述】

一行，表示该串正整数中包含的满足 $A-B=C$ 的数对的个数。

【示例一】

输入：

```
4 1
1 1 2 3
```

输出：

```
3
```

【解法】

由于顺序不影响最终结果，所以可以先把整个数组排序。

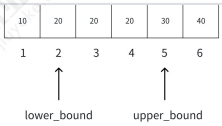
由 $A-B=C$ 得： $B=A-C$，由于 C 是已知的数，我们可以从前往后枚举所有的 A，然后去前面找有多少个符合要求的 B，正好可以用二分快速查找出区间的长度。

【STL 的使用】

1. $lower_bound$：传入要查询区间的左右迭代器（注意是左闭右开的区间，如果是数组就是左右指针）以及要查询的值 k，然后返回该数组中 $\geq k$ 的第一个位置；
2. $upper_bound$：传入要查询区间的左右迭代器（注意是左闭右开的区间，如果是数组就是左右指针）以及要查询的值 k，然后返回该数组中 $>k$ 的第一个位置；

比如： $a=[10,20,20,20,30,40]$，设下标从 1 开始计数，在整个数组中查询 20：

1. $lower_bound(a+1,a+1+6,20)$，返回 $a+2$ 位置的指针；
2. $upper_bound(a+1,a+1+6,20)$，返回 $a+5$ 位置的指针；
3. 然后两个指针相减，就是包含 20 这个数区间的长度。



【注意】

- 虽然 STL 用起来很爽，但是不要依赖它。因为后续学习「二分答案」的时候， STL 就无法使用了；
- 同时 STL 的使用范围很「局限」，查询「有序序列」的时候才有用，数组无序的时候就无法使用。但是我们的二分模板也能在「数组无序」的时候使用，只要有「二段性」即可。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

typedef long long LL;

const int N = 2e5 + 10;

LL n, c;
LL a[N];

int main()
{
    cin >> n >> c;
    for(int i = 1; i <= n; i++) cin >> a[i];

    sort(a + 1, a + 1 + n);

    LL ret = 0;
    for(int i = 2; i <= n; i++)
    {
        // a[i]
        LL b = a[i] - c;
        ret += upper_bound(a + 1, a + i, b) - lower_bound(a + 1, a + i, b);
    }

    cout << ret << endl;

    return 0;
}
```

7.1.3 烦恼的高考志愿

题目来源： 洛谷

题目链接： P1678 烦恼的高考志愿

难度系数： ★★

【题目描述】

现有 m 所学校，每所学校预计分数线是 a_i。有 n 位学生，估分分别为 b_i。

> 根据 n 位学生的估分情况，分别给每位学生推荐一所学校，要求学校的预计分数线和学生的估分相差最小（可高可低，毕竟是估分嘛），这个最小值为不满意度。求所有学生不满意度和的最小值。

【输入描述】

> 第一行读入两个整数 m, n。m 表示学校数，n 表示学生数。
>
> 第二行共有 m 个数，表示 m 个学校的预计录取分数。第三行有 n 个数，表示 n 个学生的估分成绩。
>
> 对于 100% 的数据，$1 \leq n, m \leq 10^5$，估分和录取线 $\leq 10^6$ 且均为非负整数。

【输出描述】

> 输出一行，为最小的不满度之和。

【示例一】

> 输入：
>
> 4 3
>
> 513 598 567 689
>
> 500 600 550
>
> 输出：
>
> 32

【解法】

先把学校的录取分数「排序」，然后针对每一个学生的成绩 b，在「录取分数」中二分出 $\geq b$ 的「第一个」位置 pos，那么差值最小的结果要么在 pos 位置，要么在 $pos-1$ 位置。

取 $abs(a[pos]-b)$ 与 $abs(a[pos-1]-b)$ 两者的「最小值」即可。

细节问题：

- 如果所有元素都大于 b 的时候，$pos-1$ 会在 0 下标的位置，有可能结果出错；
- 如果所有元素都小于 b 的时候，pos 会在 n 的位置，此时结果倒不会出错，但是我们要想到这个细节问题，这道题不出错不代表下一道题不出错。

加上两个左右护法，结果就不会出错了。

【参考代码】

代码块

```
1  #include <iostream>
2  #include <algorithm>
```

```cpp

using namespace std;

typedef long long LL;

const int N = 1e5 + 10;

int n, m;
LL a[N];

int find(LL x)
{
    int left = 1, right = m;
    while(left < right)
    {
        int mid = (left + right) / 2;
        if(a[mid] >= x) right = mid;
        else left = mid + 1;
    }
    return left;
}

int main()
{
    cin >> m >> n;
    for(int i = 1; i <= m; i++) cin >> a[i];
    sort(a + 1, a + 1 + m);

    // 加上左右护法
    a[0] = -1e7 + 10;

    LL ret = 0;
    for(int i = 1; i <= n; i++)
    {
        LL b; cin >> b;
        int pos = find(b);
        ret += min(abs(a[pos] - b), abs(a[pos - 1] - b));
    }

    cout << ret << endl;

    return 0;
}
```

7.2 二分答案

准确来说，应该叫做「二分答案 + 判断」。

二分答案可以处理大部分「最大值最小」以及「最小值最大」的问题。如果「解空间」在从小到大的「变化」过程中，「判断」答案的结果出现「二段性」，此时我们就可以「二分」这个「解空间」，通过「判断」，找出最优解。

刚接触的时候，可能觉得这个「算法原理」很抽象。没关系，3 道题的练习过后，你会发现这个「二分答案」的原理其实很容易理解，重点是如何去「判断」答案的可行性。

7.2.1 木材加工

> **题目来源：** 洛谷
>
> **题目链接：** P2440 木材加工
>
> **难度系数：** ★★

【题目描述】

> 木材厂有 n 根原木，现在想把这些木头切割成 k 段长度**均**为 l 的小段木头（木头有可能有剩余）。
>
> 当然，我们希望得到的小段木头越长越好，请求出 l 的最大值。
>
> 木头长度的单位是 cm，原木的长度都是正整数，我们要求切割得到的小段木头的长度也是正整数。
>
> 例如有两根原木长度分别为 11 和 21，要求切成等长的 6 段，很明显能切出来的小段木头长度最长为 5。

【输入描述】

> 第一行是两个正整数 n, k，分别表示原木的数量，需要得到的小段的数量。
>
> 接下来 n 行，每行一个正整数 L_i，表示一根原木的长度。
>
> 对于 100% 的数据，有 $1 \le n \le 10^5, 1 \le k \le 10^8, 1 \le L_i \le 10^8 (i \in [1, n])$。

【输出描述】

> 仅一行，即 l 的最大值。
>
> 如果连 $1cm$ 长的小段都切不出来，输出 0。

【示例一】

> 输入：
>
> ```
> 3 7
> ```

```
232
124
456
```

输出：

```
114
```

【解法】

学习「二分答案」这个算法，基本上都会把这道比较简单的题当成例题~

设要切成的长度为 x ，能切成的段数为 c 。根据题意，我们可以发现如下性质，：

- 当 x 增大的时候， c 在减小。也就是最终要切成的长度越大，能切的段数越少；
- 当 x 减小的时候， c 在增大。也就是最终要切成的长度越小，能切的段数越多。

那么在整个「解空间」里面，设最终的结果是 ret ，于是有：

- 当 $x \leq ret$ 时， $c \geq k$ 。也就是「要切的长度」小于等于「最优长度」的时候，最终切出来的段数「大于等于」 k ；
- 当 $x > ret$ 时， $c < k$ 。也就是「要切的长度」大于「最优长度」的时候，最终切出来的段数「小于」 k ；

在解空间中，根据 ret 的位置，可以将解集分成两部分，具有「二段性」，那么我们就可以「二分答案」。

当我们每次二分一个切成的长度 x 的时候，如何算出能切的段数 c ？

- 很简单，遍历整个数组，针对每一根木头，能切成的段数就是 $a[i] \ / \ x$ 。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 1e5 + 10;

LL n, k;
LL a[N];
```

```cpp

// 当切割长度为 x 的时候，最多能切出来多少段
LL calc(LL x)
{
    LL cnt = 0;
    for(int i = 1; i <= n; i++)
    {
        cnt += a[i] / x;
    }
    return cnt;
}

int main()
{
    cin >> n >> k;
    for(int i = 1; i <= n; i++) cin >> a[i];

    LL left = 0, right = 1e8;
    while(left < right)
    {
        LL mid = (left + right + 1) / 2;
        if(calc(mid) >= k) left = mid;
        else right = mid - 1;
    }

    cout << left << endl;

    return 0;
}
```

7.2.2 砍树

> **题目来源：** 洛谷
>
> **题目链接：** P1873 [COCI 2011/2012 #5] EKO / 砍树
>
> **难度系数：** ★★

【题目描述】

> 伐木工人 Mirko 需要砍 M 米长的木材。对 Mirko 来说这是很简单的工作，因为他有一个漂亮的新伐木机，可以如野火一般砍伐森林。不过，Mirko 只被允许砍伐一排树。
>
> Mirko 的伐木机工作流程如下：Mirko 设置一个高度参数 H（米），伐木机升起一个巨大的锯片到高度 H，并锯掉所有树比 H 高的部分（当然，树木不高于 H 米的部分保持不变）。Mirko 就得到树木被锯下的部分。例如，如果一排树的高度分别为 $20, 15, 10, 17$，Mirko 把锯片升到 15 米的

> 高度，切割后树木剩下的高度将是 $15, 15, 10, 15$，而 Mirko 将从第 1 棵树得到 5 米，从第 4 棵树得到 2 米，共得到 7 米木材。
>
> Mirko 非常关注生态保护，所以他不会砍掉过多的木材。这也是他尽可能高地设定伐木机锯片的原因。请帮助 Mirko 找到伐木机锯片的最大的整数高度 H，使得他能得到的木材至少为 M 米。换句话说，如果再升高 1 米，他将得不到 M 米木材。

【输入描述】

> 第 1 行 2 个整数 N 和 M，N 表示树木的数量，M 表示需要的木材总长度。
>
> 第 2 行 N 个整数表示每棵树的高度。
>
> 对于 100% 的测试数据，$1 \leq N \leq 10^6, 1 \leq M \leq 2 \times 10^9$，树的高度 $\leq 4 \times 10^5$，树的高度总和 $> M$。

【输出描述】

> 1 个整数，表示锯片的最高高度。

【示例一】

> 输入：
>
> 4 7
>
> 20 15 10 17
>
> 输出：
>
> 15

【示例二】

> 输入：
>
> 5 20
>
> 4 42 40 26 46
>
> 输出：
>
> 36

【解法】

设伐木机的高度为 H，能得到的木材为 C。根据题意，我们可以发现如下性质，：

- 当 H 增大的时候，C 在减小；
- 当 H 减小的时候，C 在增大。

那么在整个「解空间」里面，设最终的结果是 ret，于是有：

- 当 $H \leq ret$ 时，$C \geq M$。也就是「伐木机的高度」大于等于「最优高度」时，能得到的木材「小于等于」 M；

- 当 $H > ret$ 时， $C < M$。也就是「伐木机的高度」小于「最优高度」时，能得到的木材「大于」 M。

在解空间中，根据 ret 的位置，可以将解集分成两部分，具有「二段性」，那么我们就可以「二分答案」。

当我们每次二分一个伐木机的高度 H 的时候，如何算出得到的木材 C？

- 很简单，遍历整个数组，针对每一根木头，能切成的木材就是 $a[i] - H$。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 1e6 + 10;

LL n, m;
LL a[N];

// 当伐木机的高度为 x 时，所能获得的木材
LL calc(LL x)
{
    LL ret = 0;
    for(int i = 1; i <= n; i++)
    {
        if(a[i] > x) ret += a[i] - x;
    }
    return ret;
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];

    LL left = 1, right = 2e9;
    while(left < right)
    {
        LL mid = (left + right + 1) / 2;
        if(calc(mid) >= m) left = mid;
```

```
33            else right = mid - 1;
34        }
35
36        cout << left << endl;
37
38        return 0;
39    }
```

7.2.3 跳石头

> **题目来源：** 洛谷
>
> **题目链接：** P2678 [NOIP2015 提高组] 跳石头
>
> **难度系数：** ★★

【题目描述】

> 一年一度的“跳石头”比赛又要开始了！
>
> 这项比赛将在一条笔直的河道中进行，河道中分布着一些巨大岩石。组委会已经选择好了两块岩石作为比赛起点和终点。在起点和终点之间，有 N 块岩石（不含起点和终点的岩石）。在比赛过程中，选手们将从起点出发，每一步跳向相邻的岩石，直至到达终点。
>
> 为了提高比赛难度，组委会计划移走一些岩石，使得选手们在比赛过程中的最短跳跃距离尽可能长。由于预算限制，组委会至多从起点和终点之间移走 M 块岩石（不能移走起点和终点的岩石）。
>
> 对于 100% 的数据， $0 \le M \le N \le 50000, 1 \le L \le 1000000000$ 。

【输入描述】

> 输入文件第一行包含三个整数 L, N, M ，分别表示起点到终点的距离，起点和终点之间的岩石数，以及组委会至多移走的岩石数。
>
> 接下来 N 行，每行一个整数，第 i 行的整数 $D_i(0 < D_i < L)$ 表示第 i 块岩石与起点的距离。这些岩石按与起点距离从小到大的顺序给出，且不会有两个岩石出现在同一个位置。

【输出描述】

> 一个整数，即最短跳跃距离的最大值。

【示例一】

> 输入：
>
> 25 5 2
>
> 2
>
> 11

```
14
17
21
```

输出：

```
4
```

【解法】

设每次跳的最短距离是 x ，移走的石头块数为 c 。根据题意，我们可以发现如下性质：

- 当 x 增大的时候， c 也在增大；
- 当 x 减小的时候， c 也在减小。

那么在整个「解空间」里面，设最终的结果是 ret ，于是有：

- 当 $x \le ret$ 时， $c \le M$ 。也就是「每次跳的最短距离」小于等于「最优距离」时，移走的石头块数「小于等于」 M ；
- 当 $x > ret$ 时， $c > M$ 。也就是「每次跳的最短距离」大于「最优距离」时，移走的石头块数「大于」 M 。

在解空间中，根据 ret 的位置，可以将解集分成两部分，具有「二段性」，那么我们就可以「二分答案」。

当我们每次二分一个最短距离 x 时，如何算出移走的石头块数 c ？

- 定义前后两个指针 i, j 遍历整个数组，设 $i \le j$ ，每次 j 从 i 的位置开始向后移动；
- 当第一次发现 $a[j] - a[i] \ge x$ 时，说明 $[i+1, j-1]$ 之间的石头都可以移走；
- 然后将 i 更新到 j 的位置，继续重复上面两步。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 5e4 + 10;

LL l, n, m;
LL a[N];
```

```cpp

// 当最短跳跃距离为 x 时，移走的岩石数目
LL calc(LL x)
{
    LL ret = 0;
    for(int i = 0; i <= n; i++)
    {
        int j = i + 1;
        while(j <= n && a[j] - a[i] < x) j++;
        ret += j - i - 1;
        i = j - 1;
    }
    return ret;
}

int main()
{
    cin >> l >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];
    a[n + 1] = l;
    n++;

    LL left = 1, right = l;
    while(left < right)
    {
        LL mid = (left + right + 1) / 2;
        if(calc(mid) <= m) left = mid;
        else right = mid - 1;
    }

    cout << left << endl;

    return 0;
}
```